In the introduction to this section I said that `memory_order_consume` was part of the acquire-release ordering model, but it was conspicuously absent from the preceding description. This is because `memory_order_consume` is special: it's all about data dependencies, and it introduces the data-dependency nuances to the inter-thread happens-before relationship mentioned in section 5.3.2.

There are two new relations that deal with data dependencies: *dependency-ordered-before* and *carries-a-dependency-to*. Just like sequenced-before, carries-a-dependency-to applies strictly within a single thread and essentially models the data dependency between operations; if the result of an operation A is used as an operand for an operation B, then A carries-a-dependency-to B. If the result of operation A is a value of a scalar type such as an `int`, then the relationship still applies if the result of A is stored in a variable, and that variable is then used as an operand for operation B. This operation is also transitive, so if A carries-a-dependency-to B, and B carries-a-dependency-to C, then A carries-a-dependency-to C.

On the other hand, the dependency-ordered-before relationship can apply between threads. It's introduced by using atomic load operations tagged with `memory_order_consume`. This is a special case of `memory_order_acquire` that limits the synchronized data to direct dependencies; a store operation A tagged with `memory_order_release`, `memory_order_acq_rel`, or `memory_order_seq_cst` is dependency-ordered-before a load operation B tagged with `memory_order_consume` if the consume reads the value stored. This is as opposed to the synchronizes-with relationship you get if the load uses `memory_order_acquire`. If this operation B then carries-a-dependency-to some operation C, then A is also dependency-ordered-before C.

This wouldn't actually do you any good for synchronization purposes if it didn't affect the inter-thread happens-before relation, but it does: if A is dependency-ordered-before B, then A also inter-thread happens-before B.

One important use for this kind of memory ordering is where the atomic operation loads a pointer to some data. By using `memory_order_consume` on the load and `memory_order_release` on the prior store, you ensure that the pointed-to data is correctly synchronized, without imposing any synchronization requirements on any other nondependent data. The following listing shows an example of this scenario.

Listing 5.10. Using `std::memory_order_consume` to synchronize data